

```

        target.add(phone, name);
        System.out.println("c=repository, s=add, r=void");
    }

    public String find(Long phone) {
        System.out.println("c=repository, s=find, a=" + phone);
        String name = target.find(phone);
        System.out.println("c=repository, s=find, r=" + name);
        return name;
    }
}

```

Separation of concerns

Recollect that we had an *ObjectFactory*, which was supplying the providers of various interfaces to the consumers. Earlier, the factory was supplying *InMemoryUserRepository* for the *UserRepository* interface. However, the new design has two implementations for the *UserRepository* interface. One is *InMemoryUserRepository*, which offers the core functionality and the other is *UserRepositoryJournalProxy*, which offers the pluggable functionality of journaling.

The *ObjectFactory* provides which of these two to the consumers?

The answer is simple! A proxy cannot exist without a target, whereas a target has no such dependency. In other words, a core implementation of a *UserRepository* such as *InMemoryUserRepository* is always required. Along with the core provider, a proxy is required only when the customer chooses to enable a specific feature.

The responsibility of supplying the core provider with or without a proxy still rests with the *ObjectFactory*. The factory makes the decision based on some sort of configuration. Here is an example for configuration in the form of a 'properties' file.

```

user.repository=com.glarimy.ums.InMemoryUserRepository
user.repository.proxy=com.glarimy.ums.
UserRepositoryJournalProxy

```

In the above sample configuration, the *InMemoryUserRepository* is chosen as a core module against the key *user.repository*. In the second line, which is optional, the *UserRepositoryJournalProxy* is chosen as the proxy.

Our *ObjectFactory* reads the configuration from the *properties* file. It first builds the provider based on the key. Then, it checks if a proxy is also indicated in the configuration. If no proxy is indicated, it returns only the core provider. However, in case a proxy is indicated, it builds the proxy, injects the provider into it, and then returns it. We know that a proxy cannot exist without a target. In other words, the consumer always receives a proxy along with the provider, when a proxy is configured.

The factory used the following logic to supply the core providers, in the earlier implementation:

```

String className = props.getProperty(key);
Class<?> claz = Class.forName(className);
Object o = null;

try {
    o = claz.newInstance();
} catch (IllegalAccessException | InstantiationException e) {
    Singleton singleton = claz.getAnnotation(Singleton.
class);
    if (singleton != null) {
        String getter = singleton.getter();
        o = claz.getMethod(getter).invoke(claz);
    } else {
        o = claz.getMethod("getInstance").invoke(claz);
    }
}
}

```

We now add the following logic to the factory method, to wire an optional proxy to the requested object:

```

String proxyClassName = props.getProperty(key + ".proxy");
if (proxyClassName == null)
    return o;
Class<?> proxyClass = Class.forName(proxyClassName);
return proxyClass.getConstructor(claz.getInterfaces()[0]).
newInstance(o);

```

The way you define the configuration properties and a corresponding factory is up to you. You may design the configuration in any other manner. Let's say, the following can be a configuration file based on XML:

```

<component>
    <provider>com.glarimy.ums.InMemoryUserRepository</
provider>
    <proxy>com.glarimy.ums.UserRepositoryJournalProxy</proxy>
</component>

```

Or you may use JSON, YAML, or any other configuration files. Only make sure that the *ObjectFactory* knows how to interpret the configuration file.

Before concluding the proxy pattern, let's make a few observations:

1. A proxy always acts as a frontend to a target, which is another class.
2. The interface of the proxy and the target must be the same.
3. The proxy should only handle pre-processing and post-processing.